

Taller 4 IA

Integrantes del grupo:

- Lucas Ruiz - 202321331
- Emmanuel Blanco - 202312743
- Jose Luis Parra - 202410271

Punto 2. b)

A continuación presentamos los resultados de la ejecución con forwardBFS para los escenarios SimpleRescueProblem:

Layout	Tiempo (s)	Longitud del plan	Estados expandidos
tinyBase	0,021	9 acciones	345
smallRescue	0,311	23 acciones	1427
openRescue	9,034	26 acciones	12982
mediumRescue	3,984	32 acciones	6141
cornerRescue	0,426	19 acciones	2248
narrowRescue	N/A	N/A	N/A
warehouseRescue	16,024	31 acciones	11428

Punto 2. c)

Los layouts más pequeños, tinyBase, smallRescue y cornerRescue, resultaron tener planes más cortos y menos expansión de nodos. Esto se debe en parte al tamaño del layout que no permite que haya tantos estados y que limita la cantidad de acciones.

Por otra parte también es importante notar que hay una gran diferencia en la expansión de estados de layouts con pocos obstáculos y de los layouts con varios obstáculos. En particular, podríamos fijarnos en los escenarios smallRescue y openRescue, los cuales tienen un tamaño similar, sin embargo, podemos ver que los estados expandidos en openRescue son muchos más comparados con los de smallRescue, ya que en smallRescue hay más muros, lo que limita la cantidad de estados y acciones aplicables. Mientras que en openRescue sólo existen los muros exteriores que limitan el escenario, lo que le deja al agente muchas más acciones posibles a incluir en su plan para cumplir con el objetivo.

En relación a lo anteriormente mencionado, vemos que los tiempos de ejecución están relacionados a la cantidad de estados expandidos, por lo que en escenarios donde hay más acciones aplicables (y por ende estado por explorar) el agente tiene en cuenta muchas más

alternativas para el plan que en escenarios donde los obstáculos reducen la cantidad de alternativas para el plan. En particular podemos ver que warehouse rescue es el escenario con mayor tiempo de planeación ya que el layout es grande, y, a pesar de haber algunos obstáculos que se deben sortear, siguen habiendo muchas alternativas que se pueden considerar dentro del plan. Aún así, podemos ver que en este escenario se expanden menos nodos que en openRescue, a pesar de que en este escenario tome más tiempo llegar al plan.

Punto 3. d)

d.1) Comparación de estados explorados: Forward vs. Backward

En el caso de tinyBase:

- Búsqueda hacia adelante (Forward): Explora muy pocos estados. Como el mapa es un simple "pasillo", el robot casi no tiene opciones de bifurcación. Camina directo a los suministros, luego al paciente y luego al puesto médico.
- Búsqueda hacia atrás (Backward): También explora pocos estados porque, al hacer la regresión matemática de las acciones, el mapa confina las posibilidades. Retrocede desde la meta (Rescued) hasta el estado inicial casi en línea recta.

En el resto de los layouts (mediumRescue, openRescue, etc.):

- Forward Search: Explora cientos o un par de miles de estados, pero terminará encontrando la solución. Sabe exactamente dónde está el robot y qué celdas libres tiene a su alrededor en cada momento.
- Backward Search: Explora una cantidad masiva e inmanejable de estados (cientos de miles) hasta quedarse sin memoria o tiempo.
 - ¿Por qué ocurre? En la búsqueda hacia atrás, el planificador no sabe dónde están las cosas originalmente. Si el subobjetivo es que el paciente y el robot estén en el puesto médico, el planificador simula acciones hacia atrás preguntándose: "¿De dónde pude haber traído al paciente?".
 - En un mapa abierto como openRescue, el planificador generará subobjetivos asumiendo que recogió al paciente en la celda (1,1), luego creará otra rama asumiendo que lo recogió en (1,2), otra para (5,5)... y para cada una intentará trazar un camino hacia el inicio.
 - Se crean miles de "subobjetivos fantasma" (situaciones que nunca ocurrieron ni ocurrirán en el estado inicial real). Sin heurísticas fuertes o técnicas avanzadas de poda (como detectar que no puedes estar en dos celdas a la vez), el algoritmo colapsa.

d.2) ¿En qué tipos de problemas es MÁS eficiente la búsqueda regresiva?

La búsqueda hacia atrás es superior cuando existe un "efecto embudo": el estado inicial permite muchísimas acciones (alto factor de ramificación hacia adelante), pero el objetivo es muy específico y solo se puede llegar a él de pocas formas (bajo factor de ramificación hacia atrás).

Ejemplos clásicos:

1. Demostración de teoremas: Partir de axiomas (inicio) genera infinitas fórmulas inútiles. Partir del teorema a demostrar (meta) hacia atrás restringe drásticamente las reglas aplicables.
2. Múltiples estados iniciales, un solo objetivo: Un sistema para aparcar un coche. Puede iniciar en infinitas posiciones y ángulos, pero la meta siempre es el mismo espacio delimitado.
3. Ensamblaje / Manufactura: Unir piezas sueltas al azar (forward) genera millones de combinaciones erróneas. Planear cómo desarmar un motor ya ensamblado (backward) es mucho más directo y guiado.

Punto 4a. b) Demuestre con un ejemplo concreto (usando uno de los layouts del proyecto o creando uno propio) que esta heurística es admisible, es decir, que nunca sobreestima el costo real de la solución.

Ignorar las precondiciones es un problema admisible ya que, en general, el problema que este resuelve es más fácil que el no relajado. Esta lógica es evidente y demostrable usando el layout cornerRescue. En este caso, el unsatisfied tiene los efluentes {"Rescued", "patient_0"} . El robot está en la posición (1,7), el paciente en (4,1) y puesto médico en (4,2). Sabemos entonces que las acciones ejecutadas y óptimas son 19, sin embargo para el agente la opción greedy usando la heurística sería una: rescue(robot, patient_0, (4,2)). Es por esta razón que sabemos que esta heurística es admisible pues el costo de la heurística siempre será menor a el valor óptimo del recorrido real.

Punto 4b. c) Pruebe sus implementaciones sobre los layouts de layouts/simple/ y compare el número de estados explorados de A* con cada heurística versus forwardBFS sin heurística.

Layout	Heurística nodos	Acciones
tinyBase	372	9 acciones
smallRescue	1610	23 acciones
openRescue	11924	26 acciones
mediumRescue	6851	32 acciones
cornerRescue	2470	19 acciones
narrowRescue	N/A	N/A
warehouseRescue	15,061	31 acciones

Podemos ver que A* expande más nodos que ForwardBFS, en parte porque las heurísticas relajan el problema y permiten que haya más estados explorables aparte de los estados para los cuales se cubren las precondiciones. Sin embargo, podemos ver que al final ambos algoritmos llegan a una misma cantidad de acciones a realizar, lo que implica que para los

layouts dados ForwardBFS sería mejor en cuanto a cantidad de memoria usada y tiempo de ejecución.

Punto 4b. d) ¿Cuál de las dos heurísticas es más informativa en el dominio de rescate? Justifique su respuesta con evidencia empírica.

La heurística más informativa sobre el dominio de rescate es Ignore Precondition Heuristic ya que, empíricamente probado, este necesita menos nodos expandidos para satisfacer la meta. El ejemplo es fácil con cornerRescue donde este exploró 1638 mientras que ignore delete lists explora 2470.

Punto 5

Algunas instancias de multi-rescate pueden ser computacionalmente costosas. Se recomienda comenzar con layouts pequeños. Si su implementación no puede resolver alguna instancia en tiempo razonable, explique por qué y qué optimizaciones podrían ayudar

El HTN con BFS se vuelve computacionalmente costoso en instancias multi-rescate porque la complejidad crece exponencialmente con el número de pacientes: con N pacientes el BFS debe explorar $O(M^N)$ combinaciones de refinamientos, donde M es el número de refinamientos por misión. Además, el plan final crece alrededor de 19 acciones por paciente, por lo que el límite de longitud del plan debe ajustarse proporcionalmente.

Sería mejor procesar los pacientes en orden de proximidad al puesto médico, aumentar el límite de longitud dinámicamente con $25 * \text{len}(\text{pacientes})$, y reemplazar el BFS puro por una búsqueda con preferencia que priorice refinamientos que generen planes más cortos.

Punto 6. a)

Análisis del algoritmo forwardBFS

- Complejidad Espacial: $O(a^d * n)$
- Complejidad Temporal: $O(a^d)$

En las complejidades a hace referencia al factor de ramificación, o en nuestro caso, a qué tantos estados sucesores se puede llegar desde un estado. d hace referencia a la profundidad del plan, donde efectivamente vemos que se pueden expandir hacia distintos estados por cada nivel del plan, de ahí el término a^d en ambas complejidades. n hace referencia al total de flujos, el cuál depende del tamaño del layout y de los elementos del problema (robots, pacientes, suministros médicos).

Como se observa la complejidad del algoritmo es exponencial ya que se expanden todos los estados con acciones aplicables, sin hacer ningún tipo de poda o sin usar algún tipo de heurística para encaminar hacia un camino óptimo.

- Complejidad: el algoritmo es completo porque sin importar cuánto se demore, como BFS explora todos los estados con acciones aplicables, si existe por lo menos un conjunto de acciones que permitan llegar al objetivo, forwardBFS lo hallará.
- Optimalidad: este algoritmo no siempre garantiza encontrar un plan óptimo (de hecho en la mayoría de los casos puede no encontrar el plan óptimo). Los escenarios en los que forwardBFS obtiene el óptimo sin rodeos son casos en los que solo una secuencia de acciones específicas llevan al objetivo. Si hay múltiples caminos para llegar al objetivo, BFS no tiene garantizado encontrar el plan óptimo, pero si solo hay un plan válido, ese mismo plan es el óptimo y forwardBFS lo encontrará.

Análisis del algoritmo backwardSearch

- Complejidad espacial:
En el peor caso, BFS almacena todos los nodos del último nivel: $O(b^d)$, donde b es el factor de ramificación y d la profundidad del plan.

En regresión, b depende del número de acciones relevantes para cada fuente objetivo. En el peor caso, si muchas acciones agregan fuentes del objetivo, a puede ser cercano a a (número de acciones).

Por el tamaño de cada nodo (meta parcial): almacenar un conjunto de hasta n fuentes cuesta $O(n)$, así que el espacio total es $O(n * b^d)$.
- Complejidad temporal:
BFS explora todos los nodos hasta profundidad d : $O(b^d)$

Cada expansión revisa acciones relevantes y calcula regresión con operaciones de conjuntos. Si no hay indexación, esto puede ser $O(n * a)$.

Con indexación por fuente, se acerca a $O(n * b)$ por expansión.

En el peor caso: $O(n * b^d)$, y b se aproxima a a , $O(n * a^d)$.
- Complejidad:
Sí, es completo si:
 1. El espacio de metas es finito (número finito de fuentes).
 2. Se evita repetir metas ya visitadas.
 3. Se expanden todas las metas alcanzables por regresión (BFS sin podas incorrectas).
- Optimalidad

En general, no garantiza costo mínimo si hay costos no uniformes (ahí se necesitaría otros métodos como la búsqueda de costo uniforme o A* en el espacio de metas). Puede ser óptimo en número de acciones si:

1. El costo de cada acción es uniforme (1 por acción).
2. Se usa BFS sobre metas parciales.

Análisis de Heurísticas

Heurística Ignore Preconditions:

- Complejidad Temporal
 $O(b^d)$ es para la complejidad temporal de aStar, sin embargo con la heurística este cálculo llega a ser $O(E * acciones * goal)$ donde E es el número de estados expandidos, y dado a que en cada nodo la complejidad sería $O(|acciones| * |goal|)$
- Complejidad Espacial
 $O(E)$ es la cola de prioridad y conjunto de nodos visitados.
- Complejidad:
Como esta heurística es usada con A* y fue probado que es admisible en el punto 4a.b) podemos garantizar que el uso de esta heurística siempre encontrará un plan que lleve al objetivo si ese plan existe.
- Optimalidad:
La heurística da costos con base en una versión relajada del problema, por lo que un plan del mundo relajado con costos que son menores (por no considerarse obstáculos y precondiciones) siempre generará que se tomen las acciones más óptimas. Por lo tanto A* con esta heurística también sería óptimo
- ¿Es admisible?
Sí, consultar la solución del punto 4a.b
- ¿Es consistente?
Como estamos en escenarios representados por grids en 2 dimensiones, no es difícil ver que si en lugar de seguir una ruta de costo dos (digamos seguir un bloque a la derecha y luego subir otro bloque) se sigue una ruta en diagonal (que sería la ruta que se seguiría en una versión relajada del problema) se cumple una desigualdad triangular donde el costo de la heurística es menor al costo real.

Heurística Ignore Delete Lists :

- Complejidad Temporal
Dado a que la heurística es cara por el hill climbing que hace k pasos entonces en cada nodo se queda $O(K * acciones)$ por los nodos entonces es $O(E * K * acciones)$
- Complejidad Espacial
 $O(E)$ por los nodos en cola y visitados.

- Complejidad:
Como esta heurística es usada con A* y se prueba que la heurística es admisible en una subsección más adelante, se garantiza que el uso de esta heurística siempre encontrará un plan que lleve al objetivo si ese plan existe.
- Optimalidad:
La heurística da costos con base en una versión relajada del problema, por lo que un plan del mundo relajado con costos que son menores o iguales (porque al ignorar las delete lists el problema se torna más fácil o en el peor de los casos sigue siendo igual de fácil) siempre generará que se tomen las acciones más óptimas. Por lo tanto A* con esta heurística también sería óptimo
- ¿Es admisible?
Sí, al ignorar las delete lists hay más estados posibles por explorar, y esto hace que el problema pueda tornarse más fácil o permanecer igual de fácil.
- ¿Es consistente?
Como estamos en escenarios representados por grids en 2 dimensiones, no es difícil ver que si en lugar de seguir una ruta de costo dos (digamos seguir un bloque a la derecha y luego subir otro bloque) se sigue una ruta en diagonal (que sería la ruta que se seguiría en una versión relajada del problema) se cumple una desigualdad triangular donde el costo de la heurística es menor al costo real.

Análisis de HTN

- Complejidad Temporal
build_htn_hierarchy es $O(\text{acciones}^2 * \text{celdas}^2)$
heiriarchalSerach es $O(M^n * L)$ donde M es el número de refinamientos por HLA, n es el número de pacientes y L es la longitud máxima del plan.
- Complejidad Espacial
 $O(\text{refinamientos} * L)$ que almacena las secuencias de acciones precomputadas en cada HLA.
 $O(M^n * L)$ ya que en el peor caso tiene que almacenar los planes completos.

Punto 6. B) Reflexión sobre el uso de IA

Durante el desarrollo de este taller, la IA se empleó principalmente como una herramienta de apoyo conceptual y de optimización algorítmica. En lugar de utilizarla para generar el código desde cero, la usamos como una guía para comprender a mayor profundidad lo que se requería para cada punto del taller y para refinar nuestra implementación inicial.

Un caso destacado de este uso fue en el Punto 3 (Búsqueda Regresiva o *Backward Search*). Al ejecutar nuestra primera versión en mapas más grandes que tinyBase, nos enfrentamos a un problema crítico de complejidad espacial: la explosión combinatoria del

espacio de estados agotaba rápidamente la memoria RAM de la máquina, resultando en tiempos de ejecución demasiado largos.

Al interactuar con la IA y analizar el comportamiento del algoritmo, comprendimos el concepto de los "subobjetivos fantasma". La herramienta nos ayudó a implementar estrategias avanzadas para reducir el factor de ramificación hacia atrás, tales como la introducción de **validaciones de consistencia (Mutex físicos)** y un **1-Step Lookahead** para podar ramas absurdas antes de expandirlas. Asumimos la responsabilidad de integrar estos conceptos teóricos en nuestro código base.

EVIDENCIAS

Prompt: Esta es mi implementación inicial del punto 3: Backward Search. Sin embargo, noto que el algoritmo tarda demasiado en retornar una respuesta con los layouts dados, pienso que puede ser por un alto factor de ramificación, el cual hace que el algoritmo colapse. Por favor, analiza mi código y sugiere cómo podría optimizarse para tratar de reducir este factor y que el algoritmo se demore un poco menos

Versión inicial:

```
def backwardSearch(problem: Problem) -> list[Action]:
```

```
    """
```

Versión inicial de Backward Search (Naive BFS).

Itera sobre todas las acciones sin podas ni indexación.

```
    """
```

```
    start_goal = problem.goal
```

```
    real_initial_state = problem.initial_state
```

```
    all_actions = get_all_groundings(problem.domain, problem.objects)
```

```
    # 2. Inicialización de la frontera y visitados
```

```
    frontier = Queue()
```

```
    frontier.push((start_goal, []))
```

```
    visited = set([start_goal])
```

```

while not frontier.isEmpty():

    current_goal, current_plan = frontier.pop()

    if current_goal.issubset(real_initial_state):

        return current_plan

    for action in all_actions:

        new_goal = regress(current_goal, action)

        if new_goal is not None and new_goal not in visited:

            visited.add(new_goal)

            new_plan = [action] + current_plan

            frontier.push((new_goal, new_plan))

return []

```

Version final

```

def backwardSearch(problem: Problem) -> list[Action]:

    ### Your code here ###

    start_goal = problem.goal

    # 1. Obtener todas las acciones instanciadas (grounded) del problema

    all_actions = get_all_groundings(problem.domain, problem.objects)

```

2. PRE-COMPUTACIÓN: Índice inverso y detección de predicados estáticos

```
actions_by_add = {}
```

```
dynamic_predicates = set()
```

```
for action in all_actions:
```

```
    # Indexar acciones por los flujos que agregan
```

```
    for f in action.add_list:
```

```
        actions_by_add.setdefault(f, []).append(action)
```

```
        dynamic_predicates.add(f[0])
```

```
    # Identificar qué predicados cambian durante el juego
```

```
    for f in action.del_list:
```

```
        dynamic_predicates.add(f[0])
```

```
# Función auxiliar para podar estados lógicamente imposibles (Mutex)
```

```
def is_goal_consistent(g_set: frozenset) -> bool:
```

```
    at_counts = {}
```

```
    for f in g_set:
```

```
        if f[0] == "At": # f = ("At", entidad, ubicacion)
```

```
            entidad = f[1]
```

```
            at_counts[entidad] = at_counts.get(entidad, 0) + 1
```

```
            # Si una entidad debe estar en dos lugares distintos simultáneamente, es imposible
```

```
            if at_counts[entidad] > 1:
```

```
                return False
```

```
    return True
```

3. Inicializar Frontera (Cola para BFS) y Lista de Visitados (para Subsunción)

```

frontier = Queue()

frontier.push((start_goal, [])) # Guarda tuplas de (estado_objetivo, plan_construido)

visited_goals = [start_goal] # Usamos lista en vez de set para poder iterar y comparar
subconjuntos

while not frontier.isEmpty():

    current_goal, plan = frontier.pop()

    # CONDICIÓN DE PARADA: Si el objetivo parcial ya es verdad en el mapa original
    if current_goal.issubset(problem.initial_state):

        return list(reversed(plan)) # Invertimos porque construimos el plan hacia atrás

    # Buscar acciones relevantes (solo para los fluentes que aún no están resueltos por el
    estado inicial)

    unsatisfied_fluents = current_goal - problem.initial_state

    candidate_actions = set()

    for fluent in unsatisfied_fluents:

        if fluent in actions_by_add:

            candidate_actions.update(actions_by_add[fluent])

    # Expandir la frontera

    for action in candidate_actions:

        raw_new_goal = regress(current_goal, action)

        if raw_new_goal is None:

            continue

```

```

# --- OPTIMIZACIÓN 1: Limpieza de Fluentes Estáticos (Garbage Collection) ---

cleaned_goal = set()

is_impossible = False

for f in raw_new_goal:

    if f[0] not in dynamic_predicates:

        # Es un fuente estático (ej. Adjacent, MedicalPost)

        if f in problem.initial_state:

            continue # Ya sabemos que es verdad, no lo arrastramos para ahorrar
memoria

        else:

            # Pide un fuente estático que NO existe en el mapa (ej. pared donde no la
hay)

            is_impossible = True

            break

        else:

            cleaned_goal.add(f)

    if is_impossible:

        continue

new_goal = frozenset(cleaned_goal)

# --- OPTIMIZACIÓN 2: Poda de Inconsistencias (Mutex) ---

if not is_goal_consistent(new_goal):

    continue

# --- OPTIMIZACIÓN 3: Subsunción de Estados (State Subsumption) ---

```

Si ya visitamos un estado que exige menos cosas (es subconjunto), este camino es redundante

```
is_subsumed = False
```

```
for v in visited_goals:
```

```
    if v.issubset(new_goal):
```

```
        is_subsumed = True
```

```
        break
```

```
if not is_subsumed:
```

```
    visited_goals.append(new_goal)
```

```
    frontier.push((new_goal, plan + [action]))
```

Si la cola se vacía y no encontramos solución

```
return []
```

Punto 5:

El uso de IA fue usado para la conceptualización del jerarquías. Esto pasa ya que en los primeros intentos de las funciones estaba explorando muchos nodos y no llegaba a hacer las funciones en tiempos racionales. Por esta razón se usa IA para encontrar una función `find_path` para que implemente un BFS sobre el grafo de adyacencia del mapa para encontrar la secuencia mínima de acciones Move. Este planteamiento empieza con las presentaciones pero para conectar lo teórico con lo conceptual se usa IA para cerrar esta brecha. La función la hago yo con pseudocódigo y me lo corrige y después me ayuda a formalizarlo.

función `find_path(start, end, moves_by_from):`

```
si start == end:
```

```
    retornar []
```

```
cola ← Queue()
```

```
cola.push((start, []))
```

visitados $\leftarrow$ {start}

mientras cola no esté vacía:

(celda actual, camino) $\leftarrow$ cola.pop()

para cada acción en moves_by_from[celda_actual]:

next_cell $\leftarrow$ extraer destino de action.add_list

si next_cell no está en visitados:

nuevo_camino $\leftarrow$ camino + [acción]

si next_cell == end:

retornar nuevo_camino

visitados.add(next_cell)

cola.push((next_cell, nuevo_camino))

retornar None